YikYak for Android
Documentation for Milestone 0
Team Members:

Aseem Nerlekar
Brian Temple
Munachim Akanno
Yezen Armout

Table of Contents

1. Introduction and mission statement....4
1.1 Mission....4
1.2 Intended users....4
2. Glossary....4
3. Requirements....5
3.1 System Requirements....5
3.1.1 Mobile Devices....5
3.2 User Requirements....5
3.2.1 Accounts and anonymity....5
M0-ACC-01 — Create user....5
M0-ACC-02 — Persistent session....5
M0-ACC-03 — Public identity....5
3.2.2 Location and feed....5
M0-FEED-01 — Location-based feed....5
M0-FEED-02 — Feed sorting....6
M0-FEED-03 — Feed loading....6
3.2.3 Posts....6
M0-POST-01 — Create post....6
M0-POST-02 — Geotag post....6
M0-POST-03 — Feed visibility....6
3.2.4 Comments and threaded replies....6
M0-COM-01 — Comment on post....6
M0-COM-02 — Reply to comment....6
M0-COM-03 — Comment count....6
3.2.5 Voting and downvote removal....7
M0-VOTE-01 — Single active vote....7
M0-VOTE-02 — Score update....7
M0-VOTE-03 — Removal threshold....7
4. Software architecture....7
4.1 Architectural style....7
4.2 Data-flow diagram (DFD)....9
4.3 Key data entities....9
5. Design and module decomposition....9
5.1 Module interaction: create a post (sequence diagram)....10
5.2 Module interaction: location-based feed....11
5.3 Module interaction: comments and threaded replies....11

5.4 Module categories....11
6. Tools/frameworks....12
6.1 Tools and Frameworks....12
7. Verification plan and test cases....12
7.1 Test execution record....14
8. References....14
Individual Contribution Sheet....15

1. Introduction and mission statement

1.1 Mission

The project is an Android application inspired by YikYak that supports location-based campus discussions. It aims to let users read and contribute short posts associated with a nearby community while keeping the author's identity anonymous. Users should be able to interact with posts through comments, threaded replies, and votes. The initial milestone (M0) focuses on the core loop: establish a persistent anonymous user identity, associate posts with location, show nearby posts in a feed, and support basic discussion and voting. Later milestones may add profiles, photos, notifications, moderation controls, channels, polls, trending posts, and campus events; however, these are outside the M0 scope.

1.2 Intended users

- Community member (app user): reads nearby posts and contributes anonymously by creating posts, commenting, replying, and voting.
- Development/test team: configures the backend, tests behavior, and verifies milestone acceptance.

2. Glossary

- **Anonymous posting:** A post is displayed without risk of exposing the author's identity to other users due to the use of only a username to create an account. The backend may still associate it with an internal user ID for ownership and abuse prevention.
- **Persistent user identifier:** An identifier that remains associated with the same app user across app restarts, subject to the chosen authentication/session mechanism.
- **Geolocation:** The device's estimated geographic position, represented by latitude and longitude.
- **Geotag:** Location metadata saved with a post.
- **Geofence / radius filter:** A geographic boundary used to determine which posts are eligible for a user's feed.
- **Feed:** The list of posts available to the user based on the feed's location and sort rules.
- **Post score:** The net vote score for a post, typically upvotes minus downvotes, subject to the final voting rules.
- **Threaded reply:** A reply associated with a particular parent comment, allowing a conversation to branch.
- **Repository:** A data-access component that separates UI/business logic from backend implementation.

- **Remote Procedure Call (RPC)**: A function that can be invoked by an application to execute logic on a remote server or backend. In this project, RPCs are used to perform database operations.
- **ViewModel:** A UI-layer component that holds screen state and coordinates user actions with repositories.
- **Supabase:** The backend platform configured in the project for authentication and database access.

3. Requirements

3.1 System Requirements

3.1.1 Mobile Devices

- **Minimum:** Android 7.0 (API 24)
- **Primary test environment:** Android 17 (API 37) emulator, 1080 × 2400 / 420 dpi
- **Other:** Internet access and location permission are required for location-dependent functionality.

3.2 User Requirements

3.2.1 Accounts and anonymity

M0-ACC-01 — Create user

When no valid user session exists on the device, the application shall allow the user to create an account by entering a username. No password or additional account information is required.

M0-ACC-02 — Persistent session

After a user has been created, the application shall preserve the user's authenticated session on the device. When the application is reopened and a valid session exists, the user shall continue directly to the application without creating another account.

M0-ACC-03 — Public identity

Posts and comments shall display the user's chosen username without exposing authentication credentials or internal account information.

3.2.2 Location and feed

M0-FEED-01 — Location-based feed

When location permission is available, the application shall obtain the user's current location and retrieve posts within 5 kilometers of that location.

M0-FEED-02 — Feed sorting

The feed shall allow the user to switch between ‘New’ and ‘Hot’ sorting. ‘New’ orders eligible posts by creation time from newest to oldest. ‘Hot’ prioritizes posts using the combined post score and comment count, with newer posts taking precedence when ranking values are equal.

M0-FEED-03 — Feed loading

The application shall retrieve additional eligible posts as the user progresses through the feed and display them without duplicating previously loaded posts.

3.2.3 Posts

M0-POST-01 — Create post

The user shall be able to submit a non-empty text post within the application's configured character limit.

M0-POST-02 — Geotag post

When a post is submitted, the application shall associate it with the user's current latitude and longitude before persisting it to the backend.

M0-POST-03 — Feed visibility

After a post has been successfully created, it shall be eligible to appear in the feed whenever it satisfies the active geographic and sorting conditions.

3.2.4 Comments and threaded replies

M0-COM-01 — Comment on post

A user shall be able to submit a comment associated with a selected post.

M0-COM-02 — Reply to comment

A user shall be able to reply to an existing comment. Each reply shall store the identifier of its parent comment so that parent-child relationships can be reconstructed and displayed as a threaded discussion.

M0-COM-03 — Comment count

The application shall maintain and display the total number of comments associated with each post, including both top-level comments and threaded replies.

3.2.5 Voting and downvote removal

M0-VOTE-01 — Single active vote

A user shall be able to upvote or downvote a post. Each user may have at most one active vote on a given post. Changing or removing a vote shall update the existing vote rather than create an additional vote.

M0-VOTE-02 — Score update

The displayed post score shall reflect the combined active votes associated with that post.

M0-VOTE-03 — Removal threshold

When a post reaches a score of -3, it shall be marked as removed and hidden from the public feed while remaining stored by the backend.

4. Software architecture

4.1 Architectural style

The current codebase follows a layered Android application structure:

1. **Presentation layer:** Jetpack Compose screens and reusable UI components.
2. **State/action layer:** ViewModels coordinate UI state and user actions.
3. **Repository/data layer:** interfaces and repository implementations isolate persistence and backend operations.
4. **Backend layer**: Supabase authentication and Postgres data services, with schema changes tracked under supabase/migrations.
5. **Device services:** Android permissions and Google Play Services location APIs provide device coordinates.

The Yik Yak for Android application is organized into the layers listed above and depicted in Figure 4.1. The application is based on Jetpack Compose for the user interface (screens) and uses the Backend-as-a-Service platform Supabase to handle data storage and user authentication, and to provide persistent user sessions. Notably, this is a hosted backend rather than a separate custom application server. The flow to access data from our PostgreSQL database (hosted by Supabase) is initiated by the user's interaction with the screens. This triggers UI state changes tracked by our ViewModels, which then request data through our repositories (PostRepository, VoteRepository, UserRepository, and others). The repositories use the Supabase SDK to call remote procedure call (RPC) functions in our database, which return JSON rows with the requested data or update table data as appropriate. The use of repositories is a standard extension of the Model-View-ViewModel (MVVM) design pattern. MVVM keeps our state logic separate from the user interface, and the repository extension takes this a step further by keeping data access in its own layer. Figure 4.2 shows the flow of data as it passes through different parts of our application.

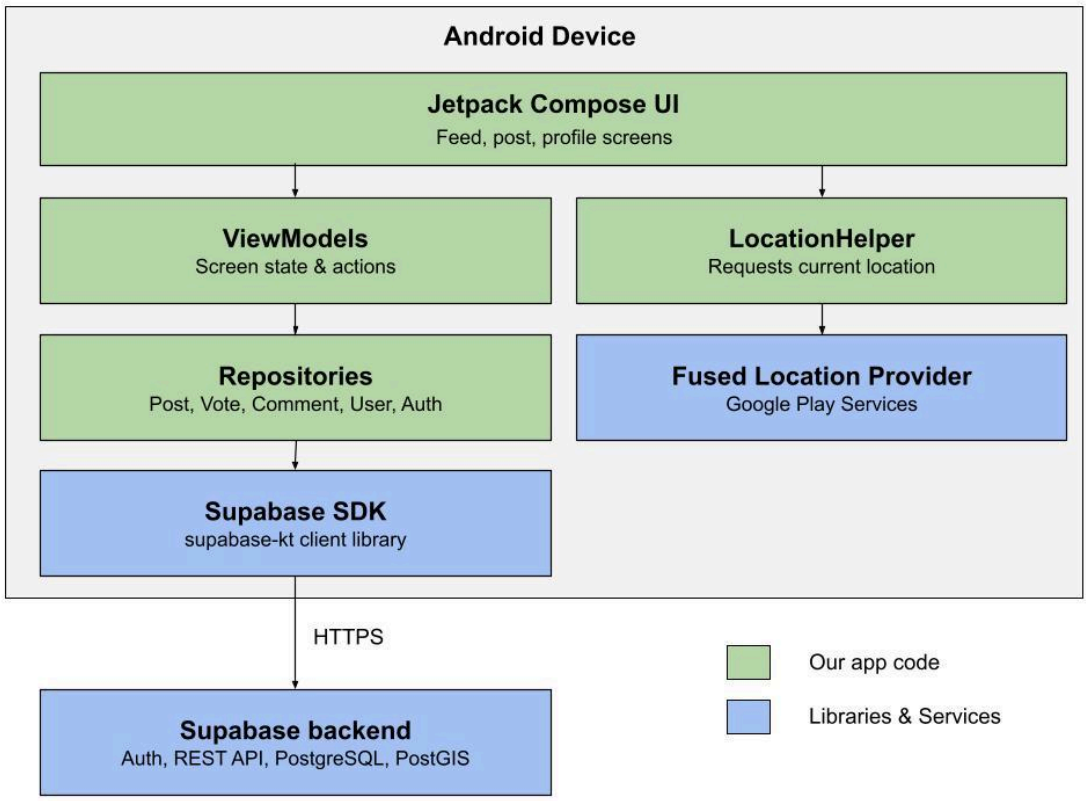


Figure 4.1: High-Level Architecture View

4.2 Data-flow diagram (DFD)

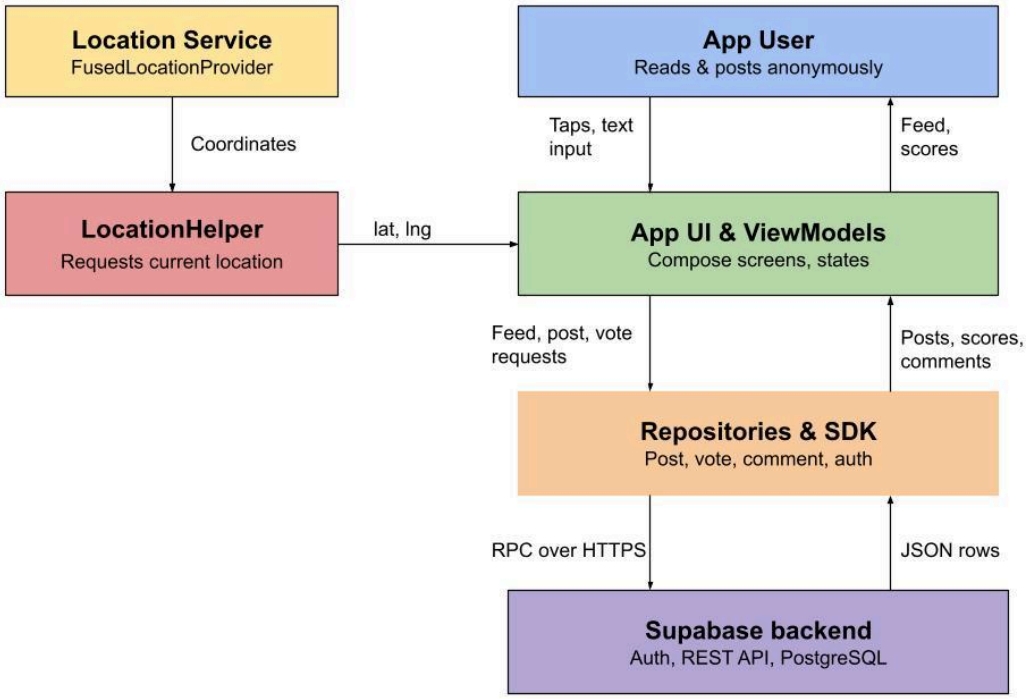


Figure 4.2: Data-flow Diagram

4.3 Key data entities

- User/session: internal stable user ID and authentication/session state.
- Post: post ID, content, creation timestamp, location coordinates, ownership metadata not exposed publicly, and current visibility/status.
- Comment: comment ID, post ID, parent comment ID (nullable for a top-level comment), content, timestamp, and ownership metadata.
- Vote: user ID, post ID, vote direction, and timestamp. A uniqueness rule should prevent more than one active vote per user/post.
- Feed query: location center, radius, selected sort, and pagination limit.

5. Design and module decomposition

The modules below reflect the repository structure and the primary responsibilities used to implement M0 functionality.

Component	Responsibility
Presentation	Compose screens for feed, post creation, post viewing, profile, and navigation
ViewModels	Maintain screen state and coordinate user actions
Repositories	Provide interfaces for posts, comments, users, authentication, and voting
DataModels	Defines application data models for posts, users, and comments that are passed between the data, state, and presentation layers.
Supabase backend	Authentication, persistent PostgreSQL data, RPCs, and database rules
Location service	Supplies device coordinates for post geotagging and feed queries
CI/Build	Gradle and GitHub Actions provide repeatable builds, linting, and tests

5.1 Module interaction: create a post (sequence diagram)

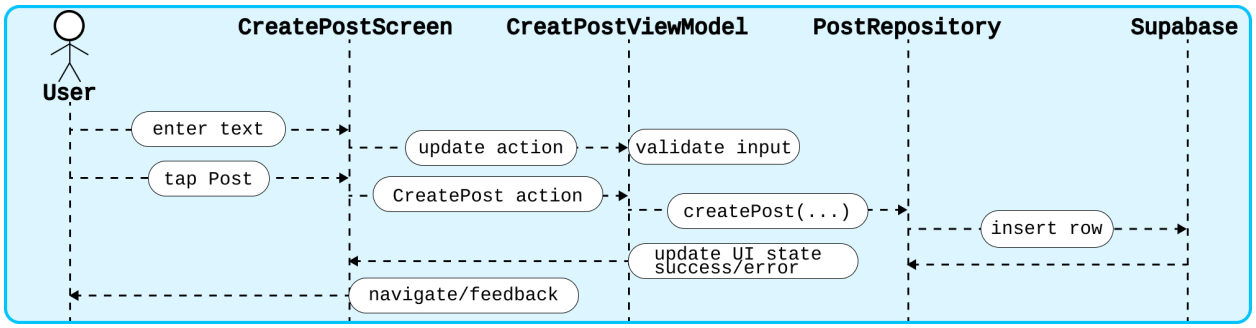


Figure 5.1: Create a Post Sequence Diagram

5.2 Module interaction: location-based feed

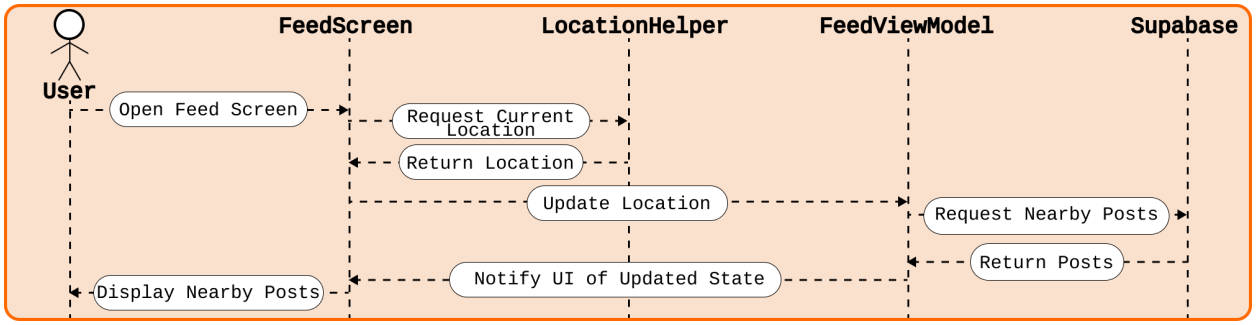


Figure 5.2: Location-Based Feed Sequence Diagram

5.3 Module interaction: comments and threaded replies

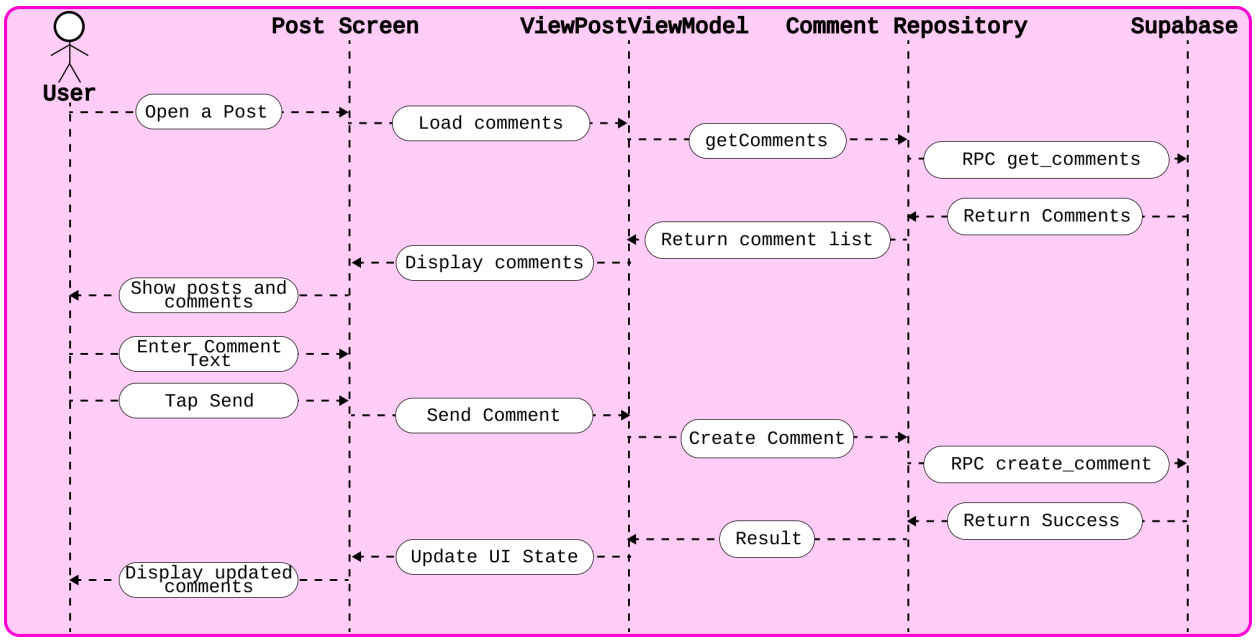


Figure 5.3: Comments and Threaded Replies Sequence Diagram

5.4 Module categories

- Boundary/UI modules: Compose screens and components; translate user actions into UI events and display state.
- Control/application modules: ViewModels; validate inputs, coordinate asynchronous work, and map results into screen state.
- Entity/model modules: Kotlin data classes for posts and users.
- Data-access modules: repository interfaces and Supabase-backed implementations.
- External adapters: Supabase client provider and location helper.

- Infrastructure/quality modules: SQL migrations, Gradle build configuration, and GitHub Actions workflow.

6. Tools/frameworks

6.1 Tools and Frameworks

The repository configuration indicates the following tools and frameworks:

- Kotlin — primary application language.
- Android SDK / Gradle Kotlin DSL — Android build system and configuration.
- Jetpack Compose and Material 3 — declarative UI.
- Navigation Compose — in-app screen navigation.
- AndroidX Lifecycle / ViewModel — UI state and lifecycle-aware work.
- Supabase Kotlin libraries — authentication and PostgreSQL data access through PostgREST and RPC functions.
- Ktor Android client — HTTP client engine used by the Supabase Kotlin integration.
- Google Play Services Location — current-location lookup.
- JUnit, AndroidX test libraries, Compose UI test support, Espresso, and coroutine test support — testing dependencies configured in Gradle.
- Supabase CLI / local Supabase configuration — repository includes supabase/config.toml, migrations, and an example local development properties file.
- GitHub Actions — workflow runs unit tests, Android lint, and production-debug APK assembly on pushes to main and pull requests.

7. Verification plan and test cases

M0 verification is organized around the application's primary user workflows. The following acceptance tests verify account creation and session persistence, geographic feed behavior, posting, threaded discussion, and voting.

Test ID	Process	Test Procedure	Expected Result
TC-ACC-01	New user creation	Launch the application on a device with no existing session, enter a valid username, and complete account creation.	A user account and authenticated session are created and the user is taken into the application.

TC-ACC-02	Existing user session	Close and reopen the application after completing TC-ACC-01.	The existing session is restored and the user continues into the application without being asked to create another account.
TC-FEED-01	Geographic feed	Open the feed with location permission enabled while test posts exist both inside and outside the configured 5 km radius.	Only eligible posts within the configured geographic region are returned and displayed.
TC-FEED-02	Feed sorting	Load a feed containing posts with different timestamps and engagement values, then switch between New and Hot.	New displays posts from newest to oldest. Hot prioritizes posts by combined score and comment count, using creation time as the tie breaker.
TC-POST-01	Create and display post	Create a valid post while location is available, then return to or refresh the feed.	The post is persisted with the submitting user's location and appears in the feed when it satisfies the geographic and sorting criteria.
TC-COM-01	Comment and threaded reply	Open a post, create a top-level comment, then reply to that comment and reply again to the resulting reply.	All comments are persisted under the correct post, replies retain the correct parent IDs, and the discussion is displayed as a thread.
TC-COM-02	Comment count	Add multiple top-level comments and replies to a post.	The displayed comment count equals the total number of top-level comments and replies associated with the post.
TC-VOTE-01	Single-user voting	Upvote a post, change the vote to a downvote, and then remove or change the vote again.	The post score changes appropriately and only one active vote from that user exists for the post at any time.

TC-VOTE-02	Downvote removal	Apply sufficient downvotes to a test post to reach the configured removal threshold.	The post becomes hidden from the public feed while its stored record remains available to the backend.

7.1 Test execution record

- Build/commit tested: cs673-yikyak — commit PR #38 - feature/user-creation on branch main
- Device/emulator and Android version: Android Medium Phone emulator, Android 17 (API 37)
- Backend environment (local/production): Local Supabase development environment
- Test date and tester: November 2nd, 2026 — Yezen Armout
- Test Breakdown:
    - TC-ACC-01 [PASS 11/02/2026]
    - TC-ACC-02 [PASS 11/02/2026]
    - TC-FEED-01 [PASS 11/02/2026]
    - TC-FEED-02 [PASS 11/02/2026]
    - TC-POST-01 [PASS 11/02/2026]
    - TC-COM-01 [PASS 11/02/2026]
    - TC-COM-02 [PASS 11/02/2026]
    - TC-VOTE-01 [PASS 11/02/2026]
    - TC-VOTE-02 [PASS 11/02/2026]
- Automated unit tests:
    - ViewPostViewModelTest (comments and replies)
    - FeedViewModelTest (feed sorting and voting)
    - CreatePostViewModelTest (post creation)
    - InMemoryUserRepositoryTest (user accounts)

8. References

- Android Developers Libraries and Wikis
- Kotlin Documentation
- Gradle Documentation
- Supabase Documentation and Wikis

Team 4 – YikYak
Team Members
Individual Contribution Sheet

Aseem Nerlekar
Brian Temple
Munachim Akanno
Yezen Armout

Summary

Between the start of the project and the Milestone 0 demo, our team held three meetings to discuss project specifications, review progress, share status updates, and make key design decisions. In addition to these meetings, we maintained consistent communication through WhatsApp to coordinate tasks, resolve questions, and keep everyone aligned on project goals and deliverables.

The project participation was divided into two tasks: research and code implementation. Code implementation was further divided by feature, with the initial feature breakdown consisting of:

- Feed page
- Comments and replies
- Create post
- User login and profile creation
- Voting
- Geofencing and geotagging

Although each team member had defined areas of responsibility, some features shared dependencies or overlapping implementation scopes. When changes to one feature affected another, we coordinated through clear communication and version control to manage modifications and maintain integration across the project.

In addition to feature-specific development, individual contributions also included setting up frameworks, initializing tools, and implementing shared infrastructure required for general application functionality or specific feature requirements. These tasks were addressed as needed to support ongoing development and integration.

1. Aseem Nerlekar (25%)

In the design and research phase of the project, prior to implementation, I researched backend infrastructure, specifically comparing two options Yezen brought up during our first group meeting: Supabase and Render. I found that Render was a cloud hosting / deployment platform that allowed for greater customization of server code but we didn't require this flexibility. Meanwhile it would require us to write our own API, WebSockets and authentication code. On the other hand Supabase provided many more built in services like anonymous user sessions, authentication and auto REST APIs generated from our tables which we could interact with using their SDK. It also had a convenient free-tier that allowed us to use a database of up to 500MB, so Supabase was chosen. Also, prior to the M0/M1 split assignment, I wrote test cases and desired outcomes of these tests for the project requirements our group had picked.

Later, I created the initial migration file with columns and constraints defined in SQL for our main tables: Posts, Comments, Votes, Profiles, Reports. I then added initial versions of core SQL functions to this schema for: creating posts, comments, votes and reports; as well as

triggers for: syncing the Profile table upon authenticated user creation, updating Post scores upon votes inserted or updated, and updating Post comment counts.

During the beginning of implementation our group decided to divide the work based on M0 requirements of which I chose the create-post functionality. For this, I created the screen for creating posts with “back” and “post” buttons and validation for checking max post length and empty text body that would enable or disable the post button. This also involved handling the UI state logic by creating the view model and the separation of the UI layer from the data layer by creating the Post repository which would hold calls to the database RPC functions using the Supabase SDK functions. Later, I added unit tests and once geotagging was implemented by Brian I incorporated it into the post creation flow.

Afterwards, I handled the voting requirement. This also involved creating the voting repository that would communicate via Supabase with the database. I housed the voting UI state logic in the existing FeedViewModel in an onVote() function. I created upvote and downvote icon buttons using a downloaded visual asset. This also involved implementing button coloring and highlighting upon click, as well as the post score indicator, the code for which is in our SocialPostCard component.

2. Brian Temple (25%)

Before the project started, I evaluated the Yik Yak mobile app’s user flow, considering screen-to-screen navigation, pop-ups, and the purpose of each screen/pop-up. Using this information, I created a user flow diagram for the Yik Yak clone application. In this user diagram, I proposed screen-to-screen navigation, pop-ups, and the purpose of each screen/pop-up.

After that, I created a mockup of the profile screen. I added a reusable profile image object, which I then implemented on the profile screen, as well as a placeholder for account karma and edit profile and settings screen buttons. I then connected these buttons to the edit profile and settings screen, which I then made UI mockups of. I finally integrated the Supabase API for use in viewing account, posts, and comments data.

Next, I researched the Fused Location Provider API for the two requirements: to feed communities based on location and to geotag posts. I added Android permissions for location services in AndroidManifest.xml, configured the specific version in libs.versions.toml allowing for baseline usage of the Fused Location Provider API. I then implemented the Fused Location Provider API in LocationHelper.kt, which uses the API to get the latitude and longitude coordinates of the phone. After that, I implemented LocationHelper.kt in GeotaggedPostScreen.kt, so that the user can click a button to get the coordinates of the phone. This implementation helps achieve the two requirements: feeding communities based on location and geotagging posts by laying the groundwork for how to properly use LocationHelper in both requirements.

Finally, I began working on and finalizing the location-based feed feature. Specifically fulfilling the requirements to show posts within a few miles of the user and sorted by hot and

new by updating SocialPostCard.kt, PostRepository.kt, FeedScreen.kt, & FeedViewModel.kt. I updated variables in SocialPostCard.kt and all related usages to reflect the Supabase SQL table, Posts. Next, I added the function getPostsWithinRadius to the Supabase database to send posts to the app when called. To cycle through the posts, I implemented feed page pagination of retrieved posts. I connected functionality to monitor user and system actions to automatically call the Supabase function getPostsWithinRadius when opening the Feed page, refreshing, the current location changes, or the next page of posts is needed. To flesh out the feed page, I updated the Feed page screen to display relevant post information such as user ID, content, and number of comments. I then implemented a button to swap between sorting the feed by recent or by a combination of the number of likes and the number of comments, and tweaked getPostsWithinRadius to perform this sorting.

3. Munachim Akanno (25%)

Before the project started, I researched and selected the project's technology stack: Kotlin, Jetpack Compose, and Gradle Kotlin DSL. I also recommended Android Studio to provide the team with an integrated and efficient development environment with the best online support for Android Development.

I initialized the Android project and implemented the first working version of the application, including basic navigation between composable screens with placeholder content. After that I developed a prototype feed screen that rendered a static post item made from a reusable PostCard UI component.

I subsequently implemented the initial user sign-on flow by creating the welcome and account creation screens and integrating their functionality with the backend. During this work, I refactored the Supabase client implementation to provide modularity and ensure a shared client instance, avoiding unnecessary client recreation across Supabase-dependent operations.

Outside these features, I initialized the GitHub and managed the build quality by creating a GitHub continuous integration (CI) action that builds the productionDebug version of the build. This was meant to serve as a check for stable builds by running some vital unit tests (without some integration tests that are best run locally) to ensure that all PRs attempting to merge were stable versions of the build. I also created PR templates for more detailed and formal Pull Requests. Additionally, I established branch protection by preventing force merges to main and requiring at least one teammate review each PR and running the GitHub CI action on every PR so the author can confirm the state of their build.

I also handled part of the supabase migration altering tables to account for columns that we missed during initial setup and changing user permissions for different actions to comply with the principle of least privilege. I also made the build support local testing at runtime to avoid

filling up the remote supabase with test data and allow teammates to locally test with the help of supabase cli further ensuring that the latest version on GitHub was a stable build.

4. Yezen Armout (25%)

Before implementation, I organized our ranking of project choices, compiled the team's Token 1 requirements into a shared document and added my own, wrote the initial draft of the M0/M1 split document, which teammates then expanded, and set up our shared Google Drive folder. During tech stack discussions, I proposed Supabase and Render as backend options, and we chose Supabase after Aseem's comparison of the two. I also scheduled the M0 demo with our TA by collecting the team's availability.

During implementation, I took ownership of the post interaction requirements: post details, commenting, replying to comments, threaded replies, and comment counts on the post detail screen. I began by building the post detail screen UI with mock data, including the Comment data model, a reusable CommentRow component, a comment input with empty-text validation, and a reply banner with a cancel option. I then implemented threaded replies in CommentThreading.kt, which orders comments so each reply appears beneath its parent, indented by depth up to four levels.

Since the comments table had no way to link a reply to its parent, I wrote a Supabase migration adding a parent_id column, updated the create_comment and get_comments RPC functions to support it, and re-applied the permissions from Munachim's hardening migration.

I then connected the screen to Supabase by creating CommentRepository and SupabaseCommentRepository to call the comment RPC functions, and ViewPostViewModel to handle loading, sending, and replying. I added a getPost function to PostRepository so the screen shows the real post, and the post and its comments reload after sending so the comment count stays accurate. I added a ViewPost navigation route and connected the feed's comments button to it. Finally, I manually tested all four post interaction requirements end to end in the emulator.

Seen and signed by:

Aseem Nerlekar	Date: 10/30/26
Brian Temple	Date: 10/30/26
Munachim Akanno	Date: 10/30/26
Yezen Armout	Date: 10130126
Glenn Merritt (TA)	Date: 10130126